گزارش آزمایشگاه سیستم‌های عامل

پروژه اول: آشنایی با هسته سیستم‌عامل xv6

کیمیا روحی مقدم - ۸۱۰۱۰۳۴۳۷
مهدیار همتی - ۸۱۰۱۰۳۵۴۹
امید طباطبائیان - ۸۱۰۱۰۳۴۶۹

پاییز ۱۴۰۵

۱ اضافه کردن پیام اعضای گروه به پیام بوت

۱.۱ شرح مسئله

هدف این بخش، تغییر سورس‌کد هسته سیستم‌عامل xv6 به گونه‌ای است که در فرآیند راه‌اندازی (Boot) سیستم‌عامل و پیش از بارگذاری و نمایش اعلان خط فرمان پوسته (Shell Prompt)، نام و شماره دانشجویی اعضای گروه در کنسول چاپ شود.

۲.۱ تحلیل فنی و محل تغییرات

سیستم‌عامل xv6 پس از اجرای کدهای راه‌انداز در `bootasm.S` و `entry.S`، وارد تابع `main()` در فایل `main.c` می‌شود. در انتهای تابع `main(void)` و بلافاصله پس از ایجاد پردازه اول کاربر (`userinit()`) و قبل از فراخوانی `mpmain()` کد چاپ اعضا قرار گرفت:

```
    userinit();      // first user process

    // ================ Boot Message ================
    cprintf("\n----------------------------------------\n");
    cprintf("Group Members:\n");
    cprintf("1. Kimia Rohi Moghadam - 810103437\n");
    cprintf("2. Mahdiyar Hemmati - 810103549\n");
    cprintf("3. Omid Tabatabaeian - 810103469\n");
    cprintf("----------------------------------------\n\n");
    // ==============================================
```

```
11
12      mpmain();        // finish this processor's setup
13
```

تغییرات اعمال‌شده در تابع `main` در فایل `main.c`

تابع `cprintf` پیام را همزمان به خروجی CGA و درگاه سریال UART ارسال می‌کند. قرار گرفتن کد درون تابع `main()` تضمین می‌کند که این پیام فقط یک بار توسط پردازنده اصلی (CPU0) چاپ شود و پردازنده‌های ثانویه آن را تکرار نکنند.

۲ قابلیت جابه‌جایی مکان‌نما در کنسول

در فایل **`console.c`**، مدیریت ورودی از طریق بافر حلقوی `input` انجام می‌پذیرد. فیلد موقعیت مکان‌نما (`cpos`) به این ساختار اضافه شد که همواره شرط $\texttt{input.w} \leq \texttt{input.cpos} \leq \texttt{input.e}$ را برآورده می‌سازد. مکان‌نمای سخت‌افزاری CGA با استفاده از ثبات‌های ۱۴ و ۱۵ درگاه CRT جابه‌جا شده و توالی‌های اسکیپ ANSI ($\leftarrow$ و $\rightarrow$) از طریق یک ماشین حالت پردازش می‌شوند.

۳ قابلیت انتخاب، رونوشت و جای‌گذاری

این قابلیت امکان تعامل پیشرفته با متن سطر جاری را از طریق کلیدهای `Ctrl+I` (شروع و پایان انتخاب)، `Ctrl+C` (کپی در حافظه موقت بدون لغو انتخاب) و `Ctrl+V` (جای‌گذاری متن کپی‌شده) فراهم می‌کند. متن انتخابی به صورت معکوس (Reverse Video / Highlight) هم در حافظه CGA با ویژگی `0x7000` و هم در ترمینال UART با کدهای `\033[7m` تا `\033[0m` برجسته می‌گردد. همچنین عملیات‌های حذف با `Backspace` و جایگزینی با فشردن کاراکتر جدید به همراه بازترسیم دقیق سطر توسط تابع `redraw_line` پیاده‌سازی شده است.

۴ پاسخ به سؤال ۲۵ (تحلیل ساختار input با GDB)

۱.۴ نقش کلی ساختار input و متغیرهای r، w و e در نسخه اولیه xv6

ساختار `input` در هسته xv6 نقش یک بافر حلقوی (Circular FIFO Buffer) با اندازه ۱۲۸ بایت (`INPUT_BUF = 128`) را ایفا می‌کند و وظیفه همگام‌سازی بر مبنای الگوی **تولیدکننده-مصرف‌کننده (Producer-Consumer)** بین وقفه‌های ورودی سخت‌افزار (در `consoleintr`) و فراخوانی‌های سیستمی خواندن فضای کاربر (در `consoleread`) را بر عهده دارد:

- **`input.r` (Read Index):** اندیس خواندن توسط پردازه‌های کاربر (مانند شل). مشخص می‌کند پردازه تا کدام خانه از بافر را خوانده است.
- **`input.w` (Write Index):** نشان‌دهنده مرز آخرین خط تاییدشده (Committed) است. تا زمانی که کلید Enter فشرده نشود، مقدار w تغییر نکرده و پردازه خواننده در حالت خواب (sleep) منتظر می‌ماند. با فشرده شدن Enter، مقدار $\texttt{w} = \texttt{e}$ شده و پردازه بیدار می‌شود (wakeup).
- **`input.e` (Edit Index):** اندیس انتهای خط در حال ویرایش. با ورود هر کاراکتر یک واحد افزایش یافته و با Backspace یک واحد کاهش می‌یابد.
- **رابطه پایستاری:**

$$\texttt{input.r} \le \texttt{input.w} \le \texttt{input.e} \quad \text{و} \quad \texttt{input.e} - \texttt{input.r} < ۱۲۸$$

۲.۴ تغییر مقادیر هنگام وارد کردن کاراکتر و فشردن Enter

فرض کنید وضعیت اولیه به صورت $r = ۰,\ w = ۰,\ e = ۰$ باشد و شل در consoleread منتظر ورودی باشد:

۱. **ورود کاراکتر `'l'`:** در خانه `input.buf[0]` قرار گرفته و $e = ۱$ می‌شود ($r = ۰,\ w = ۰,\ e = ۱$). شل همچنان خواب است.

۲. **ورود کاراکتر `'s'`:** در خانه `input.buf[1]` قرار گرفته و $e = ۲$ می‌شود ($r = ۰,\ w = ۰,\ e = ۲$).

۳. **فشردن Enter (`\n`):** در خانه `input.buf[2]` ذخیره شده، $e = ۳$ می‌شود و چون کاراکتر خط جدید دریافت شد، خط تایید می‌گردد:

$$\texttt{input.w} = \texttt{input.e} = ۳$$

سپس تابع wakeup فراخوانی می‌شود. وضعیت در این لحظه: $r = ۰,\ w = ۳,\ e = ۳$.

۴. **خواندن توسط شل:** شل بیدار شده و ۳ بایت را می‌خواند؛ به ازای هر بایت مقدار r افزایش یافته تا به w برسد:

$$r = ۳, \quad w = ۳, \quad e = ۳$$

۳.۴ State و متغیرهای جدید اضافه‌شده به پیاده‌سازی

۱. **`uint cpos`:** ثبت موقعیت دقیق مکان‌نما درون سطر جاری در بازه $[\texttt{input.w},\ \texttt{input.e}]$ جهت پشتیبانی از ویرایش درون‌خطی و درج در میانه متن.

۲. **`static int esc_state`:** ماشین حالت برای تشخیص توالی‌های اسکیپ سه بایتی ANSI (← و →) در ترمینال لینوکس/WSL.

۳. **`static char clipboard[INPUT_BUF]` و `clip_len`:** حافظه موقت کلیپ‌بورد برای نگهداری متن کپی‌شده با Ctrl+C و پیست با Ctrl+V.

۴. **`static int sel_active`:** پرچم تعیین بازه؛ با اولین Ctrl+I فعال شده و تا دومین Ctrl+I ورودی‌ها را فقط به کلیدهای مکان‌نما محدود می‌کند.

۵. **`static int has_selection`:** پرچم فعال بودن انتخاب نهایی که متن انتخابی را روی کنسول هایلایت می‌نماید.

۶. **`static uint sel_anchor`، `sel_start`، `sel_end`:** نگهداری نقاط شروع و مرزهای انتخابی جهت پشتیبانی از انتخاب مستقیم و معکوس.

۴.۴ بررسی عملی با Breakpoint در GDB

با قرار دادن نقطه توقف روی تابع consoleintr در فایل console.c:

```
(gdb) b consoleintr
Breakpoint 1 at 0x80100790: file console.c, line 235.

```

کاربر عبارت echo os را وارد کرده و با ۲ بار کلید ← مکان‌نما را روی حرف 'o' قرار می‌دهد. سپس کلمه os با دو بار Ctrl+I انتخاب و با Ctrl+C کپی می‌گردد. خروجی چاپ متغیرها در GDB به صورت زیر مشاهده شد:

```
(gdb) print input
$1 = {
  buf = "echo os", '\000' <repeats 120 times>,
  r = 0,
  w = 0,
  e = 7,
  cpos = 5
}

(gdb) print has_selection
$2 = 1

(gdb) print sel_start
$3 = 5

(gdb) print sel_end
$4 = 7

(gdb) print clipboard
$5 = "os", '\000' <repeats 125 times>
```

```
(gdb) print clip_len
$6 = 2
```

تحلیل خروجی: مقدار cpos = ۵ بازگشت صحیح مکان‌نما به حرف 'o' (اندیس ۵ رشته ۷ کاراکتری echo os) را اثبات می‌کند. همچنین مقادیر sel_start = ۵ و sel_end = ۷ و clip_len = ۲ نشان‌دهنده انتخاب و کپی دقیق کلمه os در حافظه کلیپ‌بورد همراه با فعال ماندن هایلایت (has_selection = ۱) است.

۵ سناریوهای آزمون، تست‌کیس‌های ترکیبی و دلایل انتخاب

طبق نیازمندی‌های پروژه، تمام قابلیت‌ها علاوه بر کارکرد مستقل باید به صورت ترکیبی نیز هماهنگ عمل کنند. در ادامه تست‌کیس‌های ترکیبی طراحی‌شده به همراه منطق انتخاب و نتایج آن‌ها تشریح شده است:

جدول ۱: جدول تست‌کیس‌های ترکیبی و ارزیابی جامع پیاده‌سازی

#	نام تست‌کیس	ورودی آزمون	خروجی	دلیل انتخاب (Rationale)	وضعیت
۱	کپی، حرکت و پیست در انتها	`echo cat` ← `Ctrl+I` ← ۴ بار چپ ← `Ctrl+I` ← `Ctrl+C` ← `Ctrl+E` ← `Ctrl+V`	`cat cat`	بررسی عدم تخریب نشانگر و بازه انتخابی پس از کپی، انتقال با `Ctrl+E` و تکثیر صحیح در انتهای سطر.	**پاس شد**
۲	ترکیب با قابلیت نام مستعار (Alias)	`alias hi=echo hello` ← `hi`	`hello`	اطمینان از سازگاری کامل لایه بافر کنسول با پردازش و بسط Alias پیاده‌سازی‌شده در پوسته.	**پاس شد**
۳	ترکیب با تاریخچه (History)	`echo first` ← `!!`	`first`	بررسی جای‌گیری مستقیم دستورات بازخوانی‌شده با `!!` در کنسول و امکان ویرایش مجدد آن‌ها.	**پاس شد**
۴	انتخاب معکوس و جایگزینی با رشته	`echo bad` ← `Ctrl+I` در انتها ← ۳ بار چپ ← `Ctrl+I` ← تایپ `good`	`good`	اثبات خنثی بودن جهت انتخاب (از راست به چپ) و عملکرد صحیح حذف بازه و جایگزینی متن جدید.	**پاس شد**
۵	درج و حذف در میانه سطر	`echo hXllo` ← ۴ بار چپ ← `Backspace` ← تایپ `e`	`hello`	اطمینان از صحت شیفت بافر به چپ/راست و بازترسیم صحیح کاراکترهای بعدی روی صفحه.	**پاس شد**
۶	آزمون شرایط مرزی و زیرریز (Underflow)	۵ بار چپ ← ۵ بار `Backspace` ← `Ctrl+Q/E` در خط خالی ← `echo safe`	`safe`	تضمین عدم وقوع خطای زیرریز اشاره‌گر یا Kernel Panic در رخدادهای نامعتبر ابتدای سطر.	**پاس شد**